

Actividad Obligatoria — Clase 2

Limpieza, Transformación y Análisis Exploratorio de Datos con LLMs

Objetivo

Aplicar identificación de tipos de datos, limpieza, transformación y análisis exploratorio sobre un dataset de empleados, integrando el uso de un LLM como apoyo metodológico.

Flujo del notebook

1. Clasificación de tipos de datos y criterios de manejo
2. Carga y diagnóstico del dataset
3. Limpieza (faltantes, inconsistencias, outliers)
4. Transformación (encoding, normalización, variable nueva)
5. Uso del LLM — prompts y síntesis
6. Análisis Exploratorio de Datos (EDA)
7. Conclusiones

```
In [6]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler, OrdinalEncoder
import warnings
warnings.filterwarnings('ignore')

print("Librerías importadas correctamente.")
print(f" pandas {pd.__version__}")
import sklearn; print(f" sklearn {sklearn.__version__}")
```

```
Librerías importadas correctamente.
pandas 2.3.3
sklearn 1.7.2
```

1. Clasificación de Tipos de Datos y Criterios de Manejo

Antes de cualquier transformación es fundamental entender qué representa cada columna, cuál es su tipo estadístico y cómo debe tratarse en el flujo de trabajo.

Columna	Tipo estadístico	dtype pandas esperado	Criterio de manejo	Impacto en el flujo
ID	Identificador numérico	int64	Excluir de modelos; no aporta información predictiva	Conservar solo como índice/referencia
Nombre	Categórico nominal	object	Excluir de modelos; usar como etiqueta en visualizaciones	Sin codificación
Edad	Numérico continuo	float64/int64	Corregir negativos (valor absoluto); crear variable derivada	Errores de carga pueden introducir valores imposibles
Nivel_Educativo	Categórico ordinal	object	OrdinalEncoder con orden Licenciado < Ingeniero < Doctorado	Preservar la jerarquía es crítico; OHE la destruiría
Salario	Numérico continuo	float64	Imputar faltantes con mediana; detectar outliers con IQR; normalizar	La mediana es robusta a outliers; normalizar facilita comparaciones y modelos
Categoría	Categórico nominal	object	One-Hot Encoding (drop_first=True)	Sin orden jerárquico entre Junior/Senior/Manager
Estado	Categórico nominal	object	Unificar mayúsculas (ACTIVO→Activo); luego One-Hot Encoding	Inconsistencias generan categorías artificialmente duplicadas

Criterio general: los valores numéricos requieren inspección de rango físicamente posible (edad negativa = imposible); los categóricos requieren normalización de texto antes de cualquier encoding.

```
In [8]: # Cargar el dataset
df = pd.DataFrame({
    "ID": [1, 2, 3, 4, 5],
    "Nombre": ["Juan", "María", "Pedro", "Ana", "Luis"],
    "Edad": [32, -28, 40, 35, 50],
    "Nivel_Educativo": ["Licenciado", "Ingeniero", "Doctorado", "Licenciado", "Doct
    "Salario": [50000.0, 60000.0, None, 55000.0, 200000.0],
    "Categoría": ["Junior", "Senior", "Senior", "Junior", "Manager"],
    "Estado": ["Activo", "ACTIVO", "Inactivo", "Activo", "Inactivo"]
})

print("=== Dataset original ===")
print(df.to_string())
print()
```

```

print("Tipos de datos:")
print(df.dtypes)
print()
print("Valores faltantes por columna:")
print(df.isnull().sum())
print()
print("Estadísticos descriptivos:")
df.describe()

```

=== Dataset original ===

	ID	Nombre	Edad	Nivel_Educativo	Salario	Categoría	Estado
0	1	Juan	32	Licenciado	50000.0	Junior	Activo
1	2	María	-28	Ingeniero	60000.0	Senior	ACTIVO
2	3	Pedro	40	Doctorado	NaN	Senior	Inactivo
3	4	Ana	35	Licenciado	55000.0	Junior	Activo
4	5	Luis	50	Doctorado	200000.0	Manager	Inactivo

Tipos de datos:

```

ID                int64
Nombre            object
Edad              int64
Nivel_Educativo   object
Salario           float64
Categoría         object
Estado            object
dtype: object

```

Valores faltantes por columna:

```

ID                0
Nombre            0
Edad              0
Nivel_Educativo   0
Salario           1
Categoría         0
Estado            0
dtype: int64

```

Estadísticos descriptivos:

Out[8]:

	ID	Edad	Salario
count	5.000000	5.000000	4.000000
mean	3.000000	25.800000	91250.000000
std	1.581139	30.84153	72614.851557
min	1.000000	-28.00000	50000.000000
25%	2.000000	32.00000	53750.000000
50%	3.000000	35.00000	57500.000000
75%	4.000000	40.00000	95000.000000
max	5.000000	50.00000	200000.000000

2. Limpieza de Datos

Se identificaron los siguientes problemas a corregir en orden lógico:

1. **Inconsistencias categóricas** (Estado): "ACTIVO" debe unificarse con "Activo" → `str.title()`
2. **Edad negativa** (María: -28): valor físicamente imposible → `abs()`
3. **Valor faltante en Salario** (Pedro: None): imputar con la mediana (robusta a outliers)
4. **Outlier en Salario** (Luis: 200,000): detectado con método IQR, valor >67,500 → eliminar fila

Orden de operaciones: la inconsistencia categórica y la edad se corrigen primero porque son errores de calidad independientes. La imputación precede a la detección de outliers para que el salario imputado participe en el cálculo del IQR junto con los demás valores.

```
In [11]: df_clean = df.copy()

# — Paso 1: Unificar mayúsculas en "Estado" —————
print("Estado antes de limpiar:", df_clean["Estado"].unique())
df_clean["Estado"] = df_clean["Estado"].str.title()
print("Estado después de limpiar:", df_clean["Estado"].unique())
print()

# — Paso 2: Corregir edad negativa —————
print("Edad antes:", df_clean["Edad"].tolist())
df_clean["Edad"] = df_clean["Edad"].abs()
print("Edad después:", df_clean["Edad"].tolist())
print()

# — Paso 3: Imputar salario faltante con la mediana —————
mediana_salario = df_clean["Salario"].median()
print(f"Mediana calculada sobre valores no nulos: {mediana_salario:,.0f}")
df_clean["Salario"] = df_clean["Salario"].fillna(mediana_salario)
print(f"Salario de Pedro (imputado): {df_clean.loc[df_clean['Nombre']=='Pedro', 'Salario'].tolist()}")
print()

# — Paso 4: Detección y eliminación de outliers – método IQR —————
Q1 = df_clean["Salario"].quantile(0.25)
Q3 = df_clean["Salario"].quantile(0.75)
IQR = Q3 - Q1
lim_inf = Q1 - 1.5 * IQR
lim_sup = Q3 + 1.5 * IQR

print(f"Q1 = {Q1:,.0f} | Q3 = {Q3:,.0f} | IQR = {IQR:,.0f}")
print(f"Límite inferior: Q1 - 1.5·IQR = {lim_inf:,.0f}")
print(f"Límite superior: Q3 + 1.5·IQR = {lim_sup:,.0f}")
print()

mask_outlier = (df_clean["Salario"] < lim_inf) | (df_clean["Salario"] > lim_sup)
```

```

outliers = df_clean[mask_outlier]
print(f"Outliers detectados ({len(outliers)} filas):")
print(outliers[["Nombre", "Salario"]].to_string())
print()

df_limpio = df_clean[~mask_outlier].reset_index(drop=True)
print(f"Shape antes de limpiar outliers: {df_clean.shape}")
print(f"Shape después de limpiar outliers: {df_limpio.shape}")
print()
print("Dataset limpio ")
print(df_limpio[["Nombre", "Edad", "Nivel_Educativo", "Salario", "Categoría", "Estado"]])

```

Estado antes de limpiar: ['Activo' 'ACTIVO' 'Inactivo']

Estado después de limpiar: ['Activo' 'Inactivo']

Edad antes: [32, -28, 40, 35, 50]

Edad después: [32, 28, 40, 35, 50]

Mediana calculada sobre valores no nulos: 57,500

Salario de Pedro (imputado): 57,500

Q1 = 55,000 | Q3 = 60,000 | IQR = 5,000

Límite inferior: $Q1 - 1.5 \cdot IQR = 47,500$

Límite superior: $Q3 + 1.5 \cdot IQR = 67,500$

Outliers detectados (1 filas):

	Nombre	Salario
4	Luis	200000.0

Shape antes de limpiar outliers: (5, 7)

Shape después de limpiar outliers: (4, 7)

Dataset limpio

	Nombre	Edad	Nivel_Educativo	Salario	Categoría	Estado
0	Juan	32	Licenciado	50000.0	Junior	Activo
1	María	28	Ingeniero	60000.0	Senior	Activo
2	Pedro	40	Doctorado	57500.0	Senior	Inactivo
3	Ana	35	Licenciado	55000.0	Junior	Activo

3. Transformación de Datos

Se aplican cuatro transformaciones sobre el dataset limpio:

Transformación	Columna(s)	Método	Justifica
Codificación ordinal	Nivel_Educativo	OrdinalEncoder	Existe jerar Licenciado Ingeniero (Doctorado
One-Hot Encoding	Categoría , Estado	pd.get_dummies(drop_first=True)	Sin orden; drop_fir evita

Transformación	Columna(s)	Método	Justifica
			multicoline perfecta
Normalización Min-Max	<code>Salario</code>	<code>MinMaxScaler</code> → [0, 1]	Facilita comparaci visuales e i a algoritmo sensibles a escala
Variable derivada	<code>Años_Hasta_Jubilación</code>	<code>65 - Edad</code>	Requerimie del enunci transform en tiempo restante

Nota: la normalización se aplica **después** de eliminar outliers para que los extremos del rango [min, max] sean representativos del conjunto de datos real.

```
In [13]: df_trans = df_limpio.copy()

# Codificación ordinal: Nivel_Educativo -
orden_edu = ["Licenciado", "Ingeniero", "Doctorado"]
enc_ord = OrdinalEncoder(categories=orden_edu)
df_trans["Nivel_Cod"] = enc_ord.fit_transform(df_trans[["Nivel_Educativo"]]).astype(int)
print("Codificación ordinal (Licenciado=0, Ingeniero=1, Doctorado=2):")
print(df_trans[["Nombre", "Nivel_Educativo", "Nivel_Cod"]].to_string())
print()

# One-Hot Encoding: Categoría y Estado —
df_trans = pd.get_dummies(df_trans, columns=["Categoría", "Estado"],
                           drop_first=True, dtype=int)
cols_ohe = [c for c in df_trans.columns if "Categoría_" in c or "Estado_" in c]
print("Columnas OHE generadas:", cols_ohe)
print(df_trans[["Nombre"] + cols_ohe].to_string())
print()

# Normalización Min-Max del Salario
scaler = MinMaxScaler()
df_trans["Salario_Normalizado"] = scaler.fit_transform(df_trans[["Salario"]])
print("Normalización Min-Max de Salario:")
print(df_trans[["Nombre", "Salario", "Salario_Normalizado"]].to_string())
print(f" Salario mínimo (→0.0): {scaler.data_min_[0]:.0f}")
print(f" Salario máximo (→1.0): {scaler.data_max_[0]:.0f}")
print()

# Nueva variable: Años hasta jubilación -
df_trans["Años_Hasta_Jubilación"] = 65 - df_trans["Edad"]
print("Nueva variable 'Años_Hasta_Jubilación' (= 65 - Edad):")
print(df_trans[["Nombre", "Edad", "Años_Hasta_Jubilación"]].to_string())
print()
```

```
print("Dataset final con todas las transformaciones ")
df_trans
```

Codificación ordinal (Licenciado=0, Ingeniero=1, Doctorado=2):

	Nombre	Nivel_Educativo	Nivel_Cod
0	Juan	Licenciado	0
1	María	Ingeniero	1
2	Pedro	Doctorado	2
3	Ana	Licenciado	0

Columnas OHE generadas: ['Categoría_Senior', 'Estado_Inactivo']

	Nombre	Categoría_Senior	Estado_Inactivo
0	Juan	0	0
1	María	1	0
2	Pedro	1	1
3	Ana	0	0

Normalización Min-Max de Salario:

	Nombre	Salario	Salario_Normalizado
0	Juan	50000.0	0.00
1	María	60000.0	1.00
2	Pedro	57500.0	0.75
3	Ana	55000.0	0.50

Salario mínimo (→0.0): 50,000

Salario máximo (→1.0): 60,000

Nueva variable 'Años_Hasta_Jubilación' (= 65 - Edad):

	Nombre	Edad	Años_Hasta_Jubilación
0	Juan	32	33
1	María	28	37
2	Pedro	40	25
3	Ana	35	30

Dataset final con todas las transformaciones

Out[13]:

	ID	Nombre	Edad	Nivel_Educativo	Salario	Nivel_Cod	Categoría_Senior	Estado_Inacti
--	----	--------	------	-----------------	---------	-----------	------------------	---------------

0	1	Juan	32	Licenciado	50000.0	0	0
1	2	María	28	Ingeniero	60000.0	1	1
2	3	Pedro	40	Doctorado	57500.0	2	1
3	4	Ana	35	Licenciado	55000.0	0	0



4. Uso del LLM — Prompts y Síntesis

Se utilizó **ChatGPT (GPT-4)** como apoyo metodológico en dos momentos del análisis. Se documentan las consultas, una síntesis de la respuesta y la validación propia.

Prompt 1 — Detección de outliers en datasets pequeños

Consulta enviada al LLM:

"Tengo un dataset de empleados con 5 filas y una columna Salario que incluye un valor de 200.000 mientras los demás oscilan entre 50.000 y 60.000. ¿Cómo puedo detectar y eliminar outliers usando Python y pandas? ¿IQR o z-score, cuál es más apropiado para n pequeño?"

Síntesis de la respuesta : El LLM explicó que el método IQR (rango intercuartílico) es más adecuado para muestras pequeñas porque no asume distribución normal y es resistente a los propios outliers al basarse en cuantiles. Detalló la fórmula: calcular Q1 y Q3 con `quantile()`, obtener $IQR = Q3 - Q1$, y definir los límites como $Q1 - 1.5 \cdot IQR$ (inferior) y $Q3 + 1.5 \cdot IQR$ (superior). El z-score fue desaconsejado para $n < 30$ ya que la media y la desviación estándar se distorsionan con un solo valor extremo.

Validación propia: Se verificó manualmente que con los salarios [50.000, 55.000, 57.500, 60.000, 200.000] el límite superior resulta 67.500, lo que efectivamente identifica a Luis (200.000) como outlier. El enfoque IQR fue adoptado tal como sugirió el LLM, confirmando su idoneidad para este caso.

Prompt 2 — Codificación de variables categóricas con orden

Consulta enviada al LLM:

"En mi dataset tengo una columna 'Nivel_Educativo' con valores: Licenciado, Ingeniero, Doctorado. ¿Debo usar One-Hot Encoding u OrdinalEncoder? ¿Cómo especifico el orden correcto en scikit-learn?"

Síntesis de la respuesta : El LLM explicó que One-Hot Encoding es apropiado para variables nominales (sin orden intrínseco), mientras que `OrdinalEncoder` es el método correcto cuando existe una jerarquía natural entre las categorías. Para este caso recomendó `OrdinalEncoder(categories=[['Licenciado', 'Ingeniero', 'Doctorado']])` de scikit-learn, ya que permite especificar explícitamente el orden deseado. También advirtió que usar OHE en una variable ordinal destruye la información de jerarquía, lo que puede perjudicar modelos basados en distancias o árboles de decisión.

Validación propia: Se confirmó el orden lógico: Licenciado (formación de grado) < Ingeniero (perfil técnico superior) < Doctorado (máxima formación académica). La recomendación del LLM fue adoptada íntegramente; para `Categoría` y `Estado` se aplicó OHE al ser variables nominales sin jerarquía definida.

5. Análisis Exploratorio de Datos (EDA)

Se presentan tres visualizaciones que permiten explorar distribuciones, relaciones y el efecto de la limpieza sobre el dataset.

```
In [ ]: fig = plt.figure(figsize=(16, 13))
fig.suptitle("EDA - Dataset de Empleados (Gallego, Ian)", fontsize=15, fontweight='bold')

# — Visualización 1: Boxplot Salario ANTES vs DESPUÉS de eliminar outlier —
ax1a = fig.add_subplot(3, 2, 1)
ax1b = fig.add_subplot(3, 2, 2)

ax1a.boxplot(df_clean["Salario"], vert=True, patch_artist=True,
             boxprops=dict(facecolor='lightcoral', color='darkred'),
             medianprops=dict(color='darkred', linewidth=2))
ax1a.set_title("Salario - ANTES de limpiar outlier", fontsize=11)
ax1a.set_ylabel("Salario ($)")
ax1a.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:,.0f}'))

ax1b.boxplot(df_limpio["Salario"], vert=True, patch_artist=True,
             boxprops=dict(facecolor='lightgreen', color='darkgreen'),
             medianprops=dict(color='darkgreen', linewidth=2))
ax1b.set_title("Salario - DESPUÉS de eliminar outlier", fontsize=11)
ax1b.set_ylabel("Salario ($)")
ax1b.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x:,.0f}'))

# — Visualización 2: Scatter Edad vs Salario_Normalizado —
ax2 = fig.add_subplot(3, 2, (3, 4))

colors_edu = {0: 'steelblue', 1: 'coral', 2: 'teal'}
edu_labels = {0: 'Licenciado', 1: 'Ingeniero', 2: 'Doctorado'}
for _, row in df_trans.iterrows():
    ax2.scatter(row["Edad"], row["Salario_Normalizado"],
               color=colors_edu[row["Nivel_Cod"]], s=180, zorder=5,
               label=edu_labels[row["Nivel_Cod"]])
    ax2.annotate(f' {row["Nombre"]}\n ${row["Salario"]:,.0f}',
               (row["Edad"], row["Salario_Normalizado"]),
               fontsize=9, va='center')

# Línea de tendencia
z = np.polyfit(df_trans["Edad"], df_trans["Salario_Normalizado"], 1)
p = np.poly1d(z)
x_line = np.linspace(df_trans["Edad"].min()-1, df_trans["Edad"].max()+1, 50)
ax2.plot(x_line, p(x_line), linestyle='--', color='gray', alpha=0.7, label='Tendencia')

# Leyenda sin duplicados
handles, labels = ax2.get_legend_handles_labels()
seen = {}
uniq_handles, uniq_labels = [], []
for h, l in zip(handles, labels):
    if l not in seen:
        seen[l] = True
        uniq_handles.append(h)
        uniq_labels.append(l)
ax2.legend(uniq_handles, uniq_labels, fontsize=9)
```

```

ax2.set_title("Relación entre Edad y Salario Normalizado\n(coloreado por Nivel Educativo)")
ax2.set_xlabel("Edad (años)")
ax2.set_ylabel("Salario Normalizado [0, 1]")
ax2.set_xlim(25, 45)
ax2.grid(True, alpha=0.3)

# — Visualización 3: Barplot Años hasta jubilación —————
ax3 = fig.add_subplot(3, 2, (5, 6))

bar_colors = [colors_edu[n] for n in df_trans["Nivel_Cod"]]
bars = ax3.barh(df_trans["Nombre"], df_trans["Años_Hasta_Jubilación"],
                color=bar_colors, edgecolor='white', height=0.5)

for bar, val, sal_norm in zip(bars, df_trans["Años_Hasta_Jubilación"],
                             df_trans["Salario_Normalizado"]):
    ax3.text(bar.get_width() + 0.3, bar.get_y() + bar.get_height()/2,
            f'{val} años | Sal. norm.={sal_norm:.2f}',
            va='center', fontsize=9)

from matplotlib.patches import Patch
legend_elements = [Patch(facecolor=colors_edu[k], label=v) for k, v in edu_labels.items()
                   if k in df_trans["Nivel_Cod"].values]
ax3.legend(handles=legend_elements, title="Nivel Educativo", fontsize=9)
ax3.set_title("Años hasta jubilación (65 - Edad)\n(coloreado por Nivel Educativo)",
ax3.set_xlabel("Años restantes")
ax3.set_xlim(0, 50)
ax3.grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.savefig("f:/CIENCIA DE DATOS2026/CienciadeDatos/Mineriadedatos/AO1_Clase2_Gallego_Ian.png",
            dpi=150, bbox_inches='tight')
plt.show()
print("Gráfico guardado correctamente.")

```

Interpretación de las Visualizaciones

Visualización 1 — Boxplot: efecto de la limpieza de outliers

El salario de Luis (200.000) representaba un outlier extremo: más de tres veces el valor del siguiente empleado más alto. Antes de la limpieza la mediana se ubicaba alrededor de 57.500 pero el bigote superior se extendía hasta 200.000, distorsionando cualquier análisis estadístico. Tras eliminar el outlier, el rango intercuartílico se comprime a apenas 5.000 unidades (50.000–60.000), con una distribución más homogénea y representativa del grupo de empleados.

Visualización 2 — Scatter: Edad vs Salario Normalizado

Se observa una tendencia positiva débil entre edad y salario: María (28 años, Ingeniero) tiene el mayor salario normalizado (1.0), mientras que Juan (32 años, Licenciado) tiene el menor

(0.0). Pedro (40 años, Doctorado) ocupa una posición intermedia (0.75). Con solo 4 observaciones la correlación no puede generalizarse, pero sugiere que el nivel educativo puede influir más que la edad en la determinación del salario dentro de esta muestra.

Visualización 3 — Barplot: Años hasta jubilación

Pedro es el empleado con menos tiempo restante hasta la jubilación (25 años), lo que podría ser relevante para la planificación de recursos humanos. María, pese a ser la más joven del grupo (28 años), tiene 37 años hasta la edad de retiro. Se observa que los empleados con Doctorado (Pedro) son los de mayor edad en la muestra, patrón coherente con la trayectoria académica extendida.

6. Conclusiones

Hallazgos principales

- El dataset presentaba **4 problemas de calidad** corregidos: 1 inconsistencia categórica, 1 valor imposible (edad negativa), 1 valor faltante y 1 outlier severo.
- Tras la limpieza, el **rango salarial se redujo de 150.000 a 10.000 unidades**, evidenciando cuánto distorsionaba Luis el análisis.
- La codificación ordinal confirmó que **Nivel_Educativo tiene una jerarquía clara** que debe preservarse en el modelado.
- La nueva variable **Años_Hasta_Jubilación** ofrece una perspectiva de planificación que la edad cruda no comunica directamente.